

Tutorial Sheet 1

Processes, threads and scheduling

AUTHOR

Operating Systems Teaching and Research Unit

PUBLISHED

08.05.2024

Introduction

Question 1

Consider the following piece of C code:

```
int main() {  
    fork();  
    fork();  
    exit();  
}
```

How many processes are created during the execution of this program?

i Solution

First fork: 1 parent creates 1 child

Second fork: each of the two processes create an additional process

We end up with 3 new processes created

Question 2

In a non-preemptive batch system, there are four jobs waiting to be executed with expected run times: A(9), B(6), C(3), D(5). Which scheduling algorithm should be used to minimize the average response time (latency)? What would be the optimal order for running these jobs?

i Solution

Shortest Job First minimizes the latency when the run time is known in advance:

The algorithm gives the following order: C(3), D(5), B(6), A(9)

The latency would therefore be $(3+(3+5)+(3+5+6)+(3+5+6+9))/4 = 12$

Question 3

Suppose we are operating a preemptive system using the algorithm identified in the previous question. At time $t=10$, two new processes arrive, E(2) and F(7). What is the current state of the processes? Which scheduling algorithm should be employed now to minimize the average response time? What will be the sequence of execution starting from $t=10$ to achieve this goal?

i Solution

On a preemptive system, we will use Shortest Remaining Time which minimizes the latency when the remaining time is known in advance.

On time $t=10$, C(3) + D(5) finished their execution. B has been executing for 2 units of time ($10-(3+5)$).

We are now left these tasks B(4), A(9), E(2) and F(7)

At time $t=10$, we will have a context switch because E(2) has a less remaining execution time than B(4). The order will now be E(2), B(4), F(7) and A(9)

The latency starting at $t=10$ would be $(2+(2+4)+(2+4+7)+(2+4+7+9))/4 = 10.75$

Question 4

Why, in addition to the states *ready* and *running*, is there another state called *blocked*? What is achieved by this state?

i Solution

Every program that is waiting for any IO input, (like mouse, keyboard, network, file on disk...) is marked *blocked* to make sure it is not scheduled and wasting CPU cycles

Using the `top` program in the Linux shell, you can view the processes currently managed by the kernel. It provides a real-time view, including current information about process state, CPU usage, and memory usage. On a typical Linux system, you'll notice that a large number of processes are in the *sleeping* state (corresponds to *blocked* based on our definition). This does not represent the *ready* state - which is combined with the *running* state in `top`. What do you suspect: why are so many processes in the blocked state?

i Solution

A typical Linux system would show something like

```
Tasks: 328 total,  1 running, 327 sleeping,  0 stopped,  0 zombie
```

Most processes like web browser, text editors and others are simply waiting for user input, they end up in the sleeping/blocked state

Tasks

Task 2.1: Processes and Process Creation

Write a simple C program following these specifications:

- Create and initialize a global variable to 0
- Inside a loop, each iteration will:
 - Increment the global variable

- Call `fork()`
- The child should print "Child" followed by its *pid* and the value of the global variable
- The parent should print "Parent" followed by its *pid* and the value of the global variable
- When the global variable reaches 5, exit the loop and exit the program
- Ensure that each parent and child processes continue to execute the loop until the variable reaches 5

i Solution

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_variable = 0;

int main() {
    pid_t pid;

    while (global_variable++ < 5) {

        pid = fork();
        if (pid == -1) {
            perror("fork");
            exit(EXIT_FAILURE);
        } else if (pid == 0)
            printf("Child: Global variable = %d, PID = %d\n", global_variable,
                getpid());
        else
            printf("Parent: Global variable = %d, PID = %d\n", global_variable,
                getpid());
    }

    return 0;
}
```

Question: Observe the output, how many processes are created by the end of execution? What happens if the loop doesn't stop so early?

i Solution

- 1st loop: 1 process created, total = 2
- 2nd loop: 2 processes created, total = 4
- 3rd loop: 4 processes created, total = 8
- 4th loop: 8 processes created, total = 16
- 5th loop: 16 processes created, total = 32

=> 32 processes in total, 31 created

If the loop keeps going, we would quickly saturate the number of processes in the operating system, which is usually called a *fork bomb*, and it can crash a system.

Task 2.2: Thread global variable increment

Modify the program from Task 2.1 to use threads instead of `fork()`. Use the pthread API (`pthread_create`, `pthread_exit`, `pthread_join`).

Solution

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int global_variable = 0;

void *thread_routine(void *args) {
    pthread_t current = (pthread_t)args;

    /* maximum number of threads that a thread can create, in this specific case */
    pthread_t thread_id[5];

    /* we keep track of the number of threads created within this thread */
    int nb_chlds = 0;

    while (global_variable < 5) {
        global_variable++;
        printf("Thread %lu, Global variable %d\n", current, global_variable);
        pthread_create(&thread_id[nb_chlds], NULL, thread_routine, &thread_id[nb_chlds]);
        nb_chlds++;
    }

    for (int id = 0; id < nb_chlds; id++) {
        pthread_join(thread_id[id], NULL);
    }

    return NULL;
}

int main() {
    long main_thread_id = 0;

    thread_routine(&main_thread_id);

    return 0;
}
```

Question: How many threads are created in total? Why is it different from the `fork` version?

Solution

5 threads are created. (Sometimes more threads can be created because of the lack of synchronization mechanisms).

The address space of every thread is now shared, so is the global variable. Every thread is incrementing the same variable until 5, so only five threads are created.

Task 2.3 Exec Syscall

Write a C program that performs a `fork()`. The child process should execute the `ls /` command using `exec()`, while the parent waits for the child to finish and then prints a completion message.

Here is the example how to use the `exec()` syscall:

```
int execl(const char *pathname, const char *arg, ...
          /*, (char *) NULL */);
```

```
// note that:
// - the argument after the executable should contain the
// - the last argument should be NULL
int ret = execl("/bin/ls", "ls", "/", NULL);
```

Solution

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        if (!execl("/bin/ls", "ls", "/", NULL)) {
            perror("execl\n");
        }
    } else if (pid == -1) {
        perror("fork");
    }
    wait(&pid);

    printf("\nChild exited!\n");
}
```

Write a second C program that has the same behavior, except that it will create a new thread instead of a fork. The new thread will be calling the `ls /` command with `exec()`. The main thread should be waiting for the child completion and print an exit message.

Solution

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

void *routine(void *arg) {
    if (!execl("/bin/ls", "ls", "/", NULL)) {
        perror("execl");
    }
    return NULL;
}

int main() {
    pthread_t tid;

    pthread_create(&tid, NULL, routine, NULL);
    pthread_join(tid, NULL);

    printf("\nChild exited!\n");
}
```

Questions: What differences do you observe between using `fork()` and creating a new thread (`pthread_create()`) in this context? What is the root cause of this behavior?

Solution

We notice that the main thread never prints its message (in the threaded version).

The child calls `exec` causing its address space to be replaced with the address space of the `ls` executable. Since the parent and the child share the same address space, the whole address space of the two threads is replaced as well by the code of `ls`. Since it wouldn't make sense to keep the main from executing arbitrary code, the implementation of the syscall includes the follow comment:

All threads other than the calling thread are destroyed during an `execve()`. Mutexes, condition variables, and other pthreads objects are not preserved.

Note that `execl()`, the version we used, always call `execve()`

Task 2.4: Scheduling Algorithms Implementation

For this week's assignment, you will be implementing various scheduling algorithms seen during the lecture. It will include some interactive and batch scheduler.

Begin by defining C structs that can manipulate abstract programs. Consider the necessary fields these structs will require, as well as additional structures needed by an operating system to facilitate scheduling (events, states, etc.).

Some schedulers are preemptive, what additional event would be needed?

Question:

1. Define a `struct` for representing a program (`task`). What fields should this `struct` include to manage program execution and scheduling? Provide a detailed list of essential fields and their purposes.
2. Describe subsequent data structures that an operating system would use to handle scheduling. How would this structure maintain and manage the list of ready processes efficiently? Discuss possible implementations.

Solution

```
// Scheduling-related enums
enum state {
    BLOCKED,
    RUNNING,
    READY
};

enum event {
    START,
    WAIT,
    WAKE_UP,
    TERMINATE,
    CLOCK_TICK
};

// Struct representing a program/task
struct task {
    int pid; // Process ID
    int ppid; // Parent Process ID
    int registers[20]; // Register set

    unsigned int cputime; // CPU time consumed
    unsigned int alarm; // Alarm for time-based events

    enum state state; // Current state of the task

    // Other necessary fields...
};

// Run queue to manage ready processes
// This could be implemented using a linked list or fixed-size array
```

Bonus Task: Vigenere and Monte Carlo parallelization

Re-implement the Monte Carlo pi calculation program that was implemented during the C Bootcamp and parallelize the computation using 4 threads. This involves dividing the workload among threads to perform concurrent calculations, leading to potentially faster execution compared to a single-threaded approach. You can try to parallelize Vigenere as well.